Meeami Fullstack Engineer Assignment

Objective: Build a full-stack application that supports user authentication, audio file management, cloud storage integration, and protected APIs.

Tech Stack: Python or NodeJS, MongoDB, HTML with JavaScript

- **Backend:** Python or NodeJS, MongoDB
- **Frontend:** React or HTML/CSS/JavaScript
- **Storage:** AWS S3 or Google Cloud Storage
- **Authentication:** JWT (JSON Web Tokens)
- **Containerization:** Docker & Docker Compose

Deliverables: Upon completion, share the project in a zip file via email or Google Drive. Be sure to include a readme with relevant documentation and instructions to run the solution.

Assignment Requirements:

1. **Database:** Design and implement a database containing at minimum:
    a. Users table/collection
        i. id
        ii. first_name
        iii. last_name
        iv. email
        v. hashed_password
        vi. created_at / updated_at
    b. Files table/collection
        i. id
        ii. user_id
        iii. file_type
        iv. file_name
        v. file_url (or use cloud storage reference bucket-keys)
        vi. file_metadata (duration, size, etc. as optional)

vii. created_at / updated_at

All metadata (user info, file references, timestamps, etc.) must persist in the database.

2. **Backend:** Create a backend service exposing the following APIs:
    a. POST /login
        i. Accepts email + password
        ii. Returns a JWT token on successful login
        iii. Return appropriate error on failure
    b. GET /audio (JWT Protected)
        i. Returns list of audio files associated with the authenticated user
        ii. Should include:
        iii. id
        iv. file_name
        v. file_url (signed URL or public URL)
        vi. created_at
        vii. other metadata
    c. GET /audio/:id/play (JWT Protected)
        i. Returns an audio stream OR a signed URL for playback
        ii. Must ensure users can only access their own files
    d. POST /audio/upload (JWT Protected)
        i. Accept a file upload (multipart/form-data)
        ii. Upload the file to cloud storage
        iii. Store the file reference + metadata in the database
        iv. Return success status + file details

3. **Frontend:** Build a simple frontend with the following screens:
    a. **Login Screen**
        i. Fields: email, password
        ii. On success → store JWT in frontend state
        iii. Navigate to Audio File List screen

b. **Audio File List Screen**
    i. Fetch /audio using the stored JWT
    ii. Display a list of uploaded audio files
    iii. Each entry should:
        1. show file name
        2. allow **inline audio playback** (using <audio> tag)
    iv. Button to go to Upload Audio screen

    If JWT is missing or expired → redirect back to Login screen.

c. **Upload Audio Screen**
    i. Allow user to select and upload audio files
    ii. After successful upload:
        1. redirect back to Audio File List screen
        2. refresh the list

    If JWT is missing or expired → redirect back to Login screen.

4. **Additional Notes:**
    a. All user and file metadata must be stored in the database
    b. All APIs must be JWT protected
    c. Audio files must be uploaded to cloud storage, not stored locally
    d. Only file metadata and cloud URL should be stored in the DB

<u>Deliverables:</u>

1. Database Model
    - ERD diagram or schema definition
2. Frontend–Backend Architecture Diagram
    - Clear diagram showing API flow, auth flow, and storage structure
3. Zipped Source Code
    - Full backend code (Python + DB models + API controllers + middleware)
    - Full frontend code (React or vanilla JS)
    - README with setup instructions
4. Docker Containers

- Dockerfile for backend
- Dockerfile for frontend
- docker-compose.yml to run backend, frontend, database (Postgres/Mongo)

Candidates should demonstrate:

- Clean and modular API architecture
- Correct JWT implementation
- Proper DB modelling
- Ability to integrate file uploads with cloud storage
- Basic but functional UI with real API integration
- Ability to containerize and run all components locally